

TECH

CODE REVIEW TASKS

CODE REVIEW TASKS

CODE REVIEW TASKS

20 ЗАДАНИЙ 7 ДНЕЙ КОНСПЕКТ ЛЕКЦИЙ

ИСТОЧНИК общая практика индустрии

20 ВОПРОСОВ 7 ДНЕЙ

ДЕНЬ 1 СРЕДНИЙ	HOOKS DEPS EX KEYS Q1 · Q2 · Q3	DERIVED IN EFFECT	IND	HOOKS DEPS EX KEYS	DERIVED IN EFFECT	IND
ДЕНЬ 2 СРЕДНИЙ	FORM A11Y ATION Q4 · Q5 · Q6	WS CLEANUP	API CANCELL	FORM A11Y ATION	WS CLEANUP	API CANCELL
ДЕНЬ 3 СРЕДНИЙ	REDUCER erHTML Q7 · Q8 · Q9	TS ANY	dangerouslySetInn	REDUCER erHTML	TS ANY	dangerouslySetInn
ДЕНЬ 4 СРЕДНИЙ	BIG SHARED API NO TESTS Q10 · Q11 · Q12	OVER-ABSTRACTION		BIG SHARED API NO TESTS	OVER-ABSTRACTION	
ДЕНЬ 5 ТЯЖЁЛЫЙ	FLAKY E2E HARED CONFIG Q13 · Q14 · Q15	BUNDLE REGRESSION	MF S	FLAKY E2E HARED CONFIG	BUNDLE REGRESSION	MF S
ДЕНЬ 6 СРЕДНИЙ	CI FLAGS ROLLOUT TELEMETRY Q16 · Q17 · Q18			CI FLAGS ROLLOUT TELEMETRY		
ДЕНЬ 7 СРЕДНИЙ	ERROR HANDLING PERF PR Q19 · Q20			ERROR HANDLING PERF PR		

РИТМ 30-60 МИН ТЕОРИЯ 15-30 МИН КОД В КОНСОЛИ 15 МИН ОТВЕТЫ ВСЛУХ
ДЕНЬ 7 — БЕЗ НОВОЙ ТЕОРИИ ТОЛЬКО ПРОГОН ПО 20 ВОПРОСАМ

ДЕНЬ 1 НАГРУЗКА СРЕДНИЙ 3 ВОПРОСА

HOOKS DEPS

DERIVED IN EFFECT

INDEX KEYS

ЧТО УЧИМ В ЭТОТ ДЕНЬ

HOOKS DEPS DERIVED IN EFFECT INDEX KEYS

ВОПРОСЫ ДНЯ

- Q01 Проведи review компонента с плохими hooks dependencies.
- Q02 Проведи review компонента с derived state в useEffect.
- Q03 Проведи review таблицы с index keys.

Проведи review компонента с плохими hooks dependencies.

ОПИСАНИЕ

Включить exhaustive-deps. Найти useEffect/useMemo/useCallback с пропущенными или нестабильными deps. Без них – stale closure, лишние эффекты, неработающая мемоизация.

КАК РАБОТАЕТ

- 01 eslint-plugin-react-hooks/exhaustive-deps включён.
- 02 Inline-объект в deps → новая ссылка → cycle.
- 03 ref для редко меняющихся значений.
- 04 Функциональный setState чтобы не зависеть от prev.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой PR с хуками.
- 02 Custom hooks особенно.

ПОДВОДНЫЕ КАМНИ

- 01 Игнорированный exhaustive-deps без TODO.
- 02 useCallback без правильных deps.

МОГУТ ДОСПРОСИТЬ

- 01 Когда осознанно убрать deps?
- 02 Что такое useEvent?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useEffect>

ВОПРОС Q02

Проведи review компонента с derived state в useEffect.

ОПИСАНИЕ

Anti-pattern: useEffect, который копирует props/state в локальный state. Решение — вычислять derived прямо в render через useMemo или просто выражением. useEffect нужен только для синхронизации с внешним миром.

КАК РАБОТАЕТ

- 01 Если useEffect только setState на основе props → удалить.
- 02 Заменить на useMemo / выражение в render.
- 03 Cleanup флаг или AbortController, если был.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Простые derived (sum, format).
- 02 Filter/sort из props.

ПОДВОДНЫЕ КАМНИ

- 01 Лишний effect = лишний render.
- 02 Рассинхрон derived и source.

МОГУТ ДОСПРОСИТЬ

- 01 Когда useEffect оправдан?
- 02 Чем useMemo лучше копии в state?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/learn/you-might-not-need-an-effect>

Проведи review таблицы с index keys.

ОПИСАНИЕ

key={index} ломает React при reorder/insert: state и focus уезжают в чужие строки. Заменить на стабильный id из данных. Если id нет – генерировать (uuid) при создании.

КАК РАБОТАЕТ

- 01 key={item.id} – единственно правильно для динамики.
- 02 Если данные неизменны и нет sort – index безопасен.
- 03 Math.random как key – antipattern (пересоздаёт каждый рендер).

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Sortable / filterable lists.
- 02 Drag-drop.

ПОДВОДНЫЕ КАМНИ

- 01 Index выглядит работающим, ломается при insert.
- 02 Дубли key – warning + bugs.

МОГУТ ДОСПРОСИТЬ

- 01 Когда index безопасен?
- 02 Что делать если нет id?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/learn/rendering-lists#why-does-react-need-keys>

ДЕНЬ 2 НАГРУЗКА СРЕДНИЙ 3 ВОПРОСА

FORM A11Y WS CLEANUP API CANCELLATION

ЧТО УЧИМ В ЭТОТ ДЕНЬ

FORM A11Y WS CLEANUP API CANCELLATION

ВОПРОСЫ ДНЯ

- Q04 Проведи review формы без accessible errors.
- Q05 Проведи review WebSocket клиента без cleanup.
- Q06 Проведи review API client без cancellation.

Проведи review формы без accessible errors.

ОПИСАНИЕ

Нет связи field ↔ message: screen reader не озвучивает ошибку. Добавить aria-invalid, aria-describedby → id сообщения, role="alert" для динамических ошибок, focus first invalid после submit.

КАК РАБОТАЕТ

- 01 <input aria-invalid={!err} aria-describedby={errId} />.
- 02 <p id={errId} role="alert">{err}</p>.
- 03 Цвет + иконка + текст – не только цвет.
- 04 Label для каждого input.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любая форма.
- 02 Особенно – финансовые/медицинские.

ПОДВОДНЫЕ КАМНИ

- 01 Только цвет – colorblind не видят.
- 02 Tooltip-only error недоступен с клавиатуры.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое aria-live?
- 02 Как оформить summary всех ошибок?

ПОЛНАЯ СТАТЬЯ

<https://www.w3.org/WAI/tutorials/forms/notifications/>

ВОПРОС Q05

Проведи review WebSocket клиента без cleanup.

ОПИСАНИЕ

Подписка не снимается при unmount → утечка + двойные обработчики при remount. Решение: `ws.close()` / `ws.removeEventListener` в `cleanup` `useEffect`.

КАК РАБОТАЕТ

- 01 `useEffect` → `return cleanup`.
- 02 `ws.close()` при `unmount`.
- 03 Reconnect-логика в одном месте.
- 04 `AbortController` для реакции на `cancel`.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой WS-клиент.
- 02 Real-time страницы.

ПОДВОДНЫЕ КАМНИ

- 01 Без `cleanup` – N подписок при N `mount/unmount`.
- 02 Reconnect-loop без `break`.

МОГУТ ДОСПРОСИТЬ

- 01 Как обрабатывать `reconnect`?
- 02 Зачем `backoff`?

ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Web/API/WebSocket>

Проведи review API client без cancellation.

ОПИСАНИЕ

Без AbortController стейлый запрос вернётся после unmount/смены пропов и перезапишет актуальное состояние. Решение: signal в каждый fetch + abort в cleanup.

КАК РАБОТАЕТ

- 01 AbortController на каждый запрос.
- 02 ctrl.abort() в cleanup useEffect.
- 03 AbortError ловить отдельно.
- 04 react-query это делает автоматически.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Search-typeahead.
- 02 useEffect-fetch по props.

ПОДВОДНЫЕ КАМНИ

- 01 AbortError путать с network.
- 02 Без abort – race condition.

МОГУТ ДОСПРОСИТЬ

- 01 Чем react-query упрощает?
- 02 Когда обязательна отмена?

ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Web/API/AbortController>

ДЕНЬ 3 НАГРУЗКА СРЕДНИЙ 3 ВОПРОСА

REDUCER TS ANY dangerouslySetInnerHTML

ЧТО УЧИМ В ЭТОТ ДЕНЬ

REDUCER TS ANY dangerouslySetInnerHTML

ВОПРОСЫ ДНЯ

- Q07 Проведи review reducer с impossible states.
- Q08 Проведи review TypeScript-кода с any.
- Q09 Проведи review компонента с dangerouslySetInnerHTML.

Проведи review reducer с impossible states.

ОПИСАНИЕ

State { loading; data; error } разрешает невозможные комбинации (loading=true && data). Заменить на discriminated union: { kind: 'idle' | 'loading' | 'success' | 'error' }.

КАК РАБОТАЕТ

- 01 DU состояний.
- 02 Reducer с exhaustive switch.
- 03 assertNever в default.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Async-state.
- 02 State machines.

ПОДВОДНЫЕ КАМНИ

- 01 Плоский state – невозможные комбинации.
- 02 Без exhaustive – пропуск кейса.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое impossible states unrepresentable?
- 02 Зачем assertNever?

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/narrowing.html#discriminated-unions>

ВОПРОС Q08

Проведи review TypeScript-кода с any.**ОПИСАНИЕ**

Каждый any – лазейка. Заменить на unknown + guard, конкретный тип или generic. Если без any никак – добавить TODO с причиной и пометить @ts-expect-error.

КАК РАБОТАЕТ

- 01 any → unknown + isUser-guard / Zod.
- 02 as User без проверки → runtime-баг.
- 03 Generic вместо any в helpers.
- 04 @ts-expect-error с TODO.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой TS PR.
- 02 Code-quality аудит.

ПОДВОДНЫЕ КАМНИ

- 01 any распространяется по графу.
- 02 as any маскирует реальную проблему.

МОГУТ ДОСПРОСИТЬ

- 01 Когда as оправдан?
- 02 Чем satisfies от as отличается?

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/everyday-types.html#the-any-type>

Проведи review компонента с dangerouslySetInnerHTML.

ОПИСАНИЕ

XSS-риск, если контент пользовательский и не санитизирован. Решение: DOMPurify.sanitize перед рендером, либо использовать markdown-renderer с whitelist, либо отказаться от HTML.

КАК РАБОТАЕТ

- 01 DOMPurify.sanitize(html).
- 02 Whitelist tags + attrs.
- 03 CSP с nonce/hash.
- 04 Если можно – text rendering, не innerHTML.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 User comments / posts.
- 02 Email preview.
- 03 Markdown.

ПОДВОДНЫЕ КАМНИ

- 01 Без sanitize – script injection.
- 02 Whitelist слишком широкий → onerror.

МОГУТ ДОСПРОСИТЬ

- 01 Чем CSP помогает?
- 02 Что такое stored vs reflected XSS?

ПОЛНАЯ СТАТЬЯ

<https://github.com/cure53/DOMPurify>

ДЕНЬ 4 НАГРУЗКА СРЕДНИЙ 3 ВОПРОСА

BIG SHARED API OVER-ABSTRACTION NO TESTS

ЧТО УЧИМ В ЭТОТ ДЕНЬ

BIG SHARED API OVER-ABSTRACTION NO TESTS

ВОПРОСЫ ДНЯ

Q10 Проведи review PR с большим shared component API.

Q11 Проведи review PR с over-abstraction.

Q12 Проведи review PR без тестов.

Проведи review PR с большим shared component API.

ОПИСАНИЕ

Слишком много props (15+) – компонент пытается покрыть все use-cases. Решения: compound-components, polymorphic as, headless behavior + styled-обёртки. Минимум-достаточный API.

КАК РАБОТАЕТ

- 01 Compound: `<Tabs.List><Tabs.Trigger /></Tabs>`.
- 02 Render-props / children-as-function.
- 03 as-prop для polymorphism.
- 04 Sensible defaults.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Дизайн-системы.
- 02 UI-библиотеки.

ПОДВОДНЫЕ КАМНИ

- 01 50 props → confusing API.
- 02 Тестирование становится комбинаторным.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое compound components?
- 02 Чем headless помогает?

ПОЛНАЯ СТАТЬЯ

<https://www.radix-ui.com/primitives/docs/overview/introduction>

ВОПРОС Q11

Проведи review PR с over-abstraction.

ОПИСАНИЕ

Преждевременная абстракция: generic-функция, обобщённый wrapper, factory без явной потребности. Rule of three: абстрагировать с третьего использования, не раньше. Простой явный код часто лучше.

КАК РАБОТАЕТ

- 01 Rule of three: 3 использования → абстракция.
- 02 Явные имена > generic.
- 03 Если 5+ generic – упрощать.
- 04 Wrapper, ничего не добавляющий – удалить.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Code review.
- 02 При желании 'переписать на красиво'.

ПОДВОДНЫЕ КАМНИ

- 01 DRY ради DRY.
- 02 Generic, который никто не понимает.

МОГУТ ДОСПРОСИТЬ

- 01 Чем DRY вредит?
- 02 Когда копипаста ок?

ПОЛНАЯ СТАТЬЯ

[https://en.wikipedia.org/wiki/Rule_of_three_\(computer_programming\)](https://en.wikipedia.org/wiki/Rule_of_three_(computer_programming))

Проведи review PR без тестов.

ОПИСАНИЕ

Спросить: критичная логика? Если да – blocker, нужны тесты. Если рефакторинг без новой логики – могут быть существующие. Если spike/POC – TODO + issue. Не пропускать молча.

КАК РАБОТАЕТ

- 01 Критичный код → unit/integration обязательно.
- 02 POC → TODO с issue.
- 03 Refactor → существующие тесты должны падать?
- 04 E2E для critical user paths.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой PR с логикой.
- 02 Критичные модули (auth, payment).

ПОДВОДНЫЕ КАМНИ

- 01 Approve без тестов = долг.
- 02 Тесты на implementation – хрупкие.

МОГУТ ДОСПРОСИТЬ

- 01 Какие тесты не нужны?
- 02 Когда e2e лучше unit?

ПОЛНАЯ СТАТЬЯ

<https://testing-library.com/docs/react-testing-library/intro/>

ДЕНЬ 5 НАГРУЗКА ТЯЖЁЛЫЙ 3 ВОПРОСА

FLAKY E2E**BUNDLE REGRESSION****MF SHARED CONFIG**

ЧТО УЧИМ В ЭТОТ ДЕНЬ

FLAKY E2E BUNDLE REGRESSION MF SHARED CONFIG

ВОПРОСЫ ДНЯ

Q13 Проведи review flaky e2e test.

Q14 Проведи review bundle-size regression.

Q15 Проведи review microfrontend shared dependency config.

Проведи review flaky e2e test.

ОПИСАНИЕ

Найти источник flakiness: explicit sleep вместо waitFor, гонка с анимацией, общая test data, network. Решение: locator с auto-waiting, mock network через route, изолировать данные, стабильные selectors (getByRole).

КАК РАБОТАЕТ

- 01 Заменить waitForTimeout → toHaveText/toBeVisible.
- 02 page.route для mock.
- 03 Изоляция данных между тестами.
- 04 getByRole > CSS selector.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой нестабильный e2e.
- 02 После change UI.

ПОДВОДНЫЕ КАМНИ

- 01 Hardcoded sleep – root flakiness.
- 02 Shared cookies → race.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое auto-waiting?
- 02 Чем mock network упрощает?

ПОЛНАЯ СТАТЬЯ

<https://playwright.dev/docs/best-practices>

ВОПРОС Q14

Проведи review bundle-size regression.

ОПИСАНИЕ

Bundle вырос – найти что добавилось через bundle-analyzer / git-diff package-lock. Заменить тяжёлые deps на lightweight, перенести в dynamic import, убрать unused. Установить size-limit в CI.

КАК РАБОТАЕТ

- 01 webpack-bundle-analyzer / rollup-plugin-visualizer.
- 02 git diff package-lock.json.
- 03 size-limit в CI.
- 04 lodash → lodash-es / per-method import.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 После обновления deps.
- 02 После добавления новой библиотеки.

ПОДВОДНЫЕ КАМНИ

- 01 Без analyzer – слепой поиск.
- 02 Tree-shake не работает для CommonJS.

МОГУТ ДОСПРОСИТЬ

- 01 Чем bundlephobia полезен?
- 02 Что такое side-effects: false?

ПОЛНАЯ СТАТЬЯ

<https://github.com/webpack-contrib/webpack-bundle-analyzer>

Проведи review microfrontend shared dependency config.

ОПИСАНИЕ

Несовпадение версий React между host и remote → два React-инстанса, сломанные хуки.
Проверить: shared с правильным singleton/version, одинаковые мажорные версии React, проверка на runtime.

КАК РАБОТАЕТ

- 01 shared: { react: { singleton: true, requiredVersion: '^18' } }.
- 02 Eager: false для lazy load.
- 03 Проверка React.version на runtime.
- 04 Все remotes используют одну версию.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Module Federation setup.
- 02 После обновления React.

ПОДВОДНЫЕ КАМНИ

- 01 Два React → cannot use hooks outside Component.
- 02 Version skew.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое singleton в MF?
- 02 Как проверить на runtime?

ПОЛНАЯ СТАТЬЯ

<https://module-federation.io/>

ДЕНЬ 6

НАГРУЗКА

СРЕДНИЙ

3 ВОПРОСА

CI FLAGS ROLLOUT TELEMETRY

ЧТО УЧИМ В ЭТОТ ДЕНЬ

CI FLAGS ROLLOUT TELEMETRY

ВОПРОСЫ ДНЯ

- Q16 Проведи review CI pipeline.
- Q17 Проведи review feature flag rollout.
- Q18 Проведи review frontend telemetry events.

Проведи review CI pipeline.

ОПИСАНИЕ

Чек-лист: typecheck, lint, unit, integration, e2e smoke, bundle-size, dependency-audit, build artifact, deploy. Параллелить независимые шаги. Cache node_modules + build cache. Fast feedback первым (lint/type), медленное в конце (e2e).

КАК РАБОТАЕТ

- 01 Параллель независимых jobs.
- 02 Cache npm / pnpm / yarn.
- 03 size-limit / Lighthouse CI.
- 04 Build once, deploy many.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой production-проект.
- 02 После регрессий.

ПОДВОДНЫЕ КАМНИ

- 01 Серийный pipeline без параллели – медленно.
- 02 E2E без retry – flaky.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое quality gate?
- 02 Зачем build once?

ПОЛНАЯ СТАТЬЯ

<https://docs.github.com/en/actions>

ВОПРОС Q17

Проведи review feature flag rollout.**ОПИСАНИЕ**

Чек-лист: default OFF, понятная owner, метрики до/после, kill switch, plan очистки. Не оставлять forever-flags. Тесты с обоими значениями.

КАК РАБОТАЕТ

- 01 Default = OFF.
- 02 Owner + cleanup-deadline в описании.
- 03 Метрики split-теста.
- 04 Kill switch перед production.
- 05 Тесты обеих веток.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой feature flag.
- 02 A/B-эксперименты.

ПОДВОДНЫЕ КАМНИ

- 01 Forgotten flag = долг.
- 02 Без метрик – rollout вслепую.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое gradual rollout?
- 02 Когда удалить flag?

ПОЛНАЯ СТАТЬЯ

<https://martinfowler.com/articles/feature-toggles.html>

Проведи review frontend telemetry events.

ОПИСАНИЕ

Чек-лист: понятные имена событий, документированный schema, не логируем PII, sample rate, не блокируем main thread (navigator.sendBeacon). Версионирование схемы для backward-compat.

КАК РАБОТАЕТ

- 01 naming convention (entity.action).
- 02 Schema-документация.
- 03 Без PII (email, name, address).
- 04 sendBeacon – не блокирует.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Product analytics.
- 02 Funnel-анализ.

ПОДВОДНЫЕ КАМНИ

- 01 Лог PII = GDPR violation.
- 02 fetch() блокирует unload.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое sendBeacon?
- 02 Как не пропустить event при close?

ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Web/API/Navigator/sendBeacon>

ДЕНЬ 7 НАГРУЗКА СРЕДНИЙ 2 ВОПРОСА

ERROR HANDLING PERF PR

ЧТО УЧИМ В ЭТОТ ДЕНЬ

ERROR HANDLING PERF PR

ВОПРОСЫ ДНЯ

Q19 Проведи review error handling strategy.

Q20 Проведи review performance optimization PR.

Проведи review error handling strategy.

ОПИСАНИЕ

Чек-лист: error boundaries на app + section level, structured errors (class hierarchy / DU), отчёты в Sentry, fallback UI с retry, без 'белого экрана'. Async errors отдельно от render.

КАК РАБОТАЕТ

- 01 ErrorBoundary на app + page.
- 02 Sentry / Bugsnag для отчётов.
- 03 Fallback с retry.
- 04 Структурированные ошибки (ApiError class).

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Production-приложения.
- 02 Critical paths.

ПОДВОДНЫЕ КАМНИ

- 01 Boundary в самой себе ломается.
- 02 Async errors не ловятся boundary.

МОГУТ ДОСПРОСИТЬ

- 01 Что boundary НЕ ловит?
- 02 Как обработать unhandled rejection?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/Component#catching-rendering-errors-with-an-error-boundary>

ВОПРОС Q20

Проведи review performance optimization PR.**ОПИСАНИЕ**

Спросить: есть measurement до/после? Без него – guessing. Profiler trace, Lighthouse score, bundle-size diff. Проверить, что оптимизация не вредит читаемости (преждевременная memoization).

КАК РАБОТАЕТ

- 01 Before/after metrics обязательны.
- 02 Profiler trace в PR description.
- 03 Bundle-size diff.
- 04 Не делать преждевременную мемоизацию.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Performance-PR.
- 02 После регрессий.

ПОДВОДНЫЕ КАМНИ

- 01 Без метрик – теоретическая оптимизация.
- 02 useМемо на всё → overhead.

МОГУТ ДОСПРОСИТЬ

- 01 Какие метрики смотреть?
- 02 Когда мемоизация лишняя?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/vitals>

СДАЛ ЛИ ЗАЧЁТ

ПРОЙДИ ВСЛУХ БЕЗ ПОДГЛЯДЫВАНИЙ ЕСЛИ ЗАСТРЯЛ – ВЕРНИСЬ В НУЖНЫЙ БИЛЕТ

- ☐ Найти баги hooks deps, derived in effect, index keys (Q1, Q2, Q3)
- ☐ Найти ally, cleanup, cancellation проблемы (Q4, Q5, Q6)
- ☐ Найти impossible states, any, XSS-риски (Q7, Q8, Q9)
- ☐ Найти over-engineered API, over-abstraction, отсутствие тестов (Q10, Q11, Q12)
- ☐ Разобрать flaky test, bundle regression, MF config (Q13, Q14, Q15)
- ☐ Ревьюить CI, feature flags, telemetry (Q16, Q17, Q18)
- ☐ Ревьюить error handling и performance PR (Q19, Q20)